

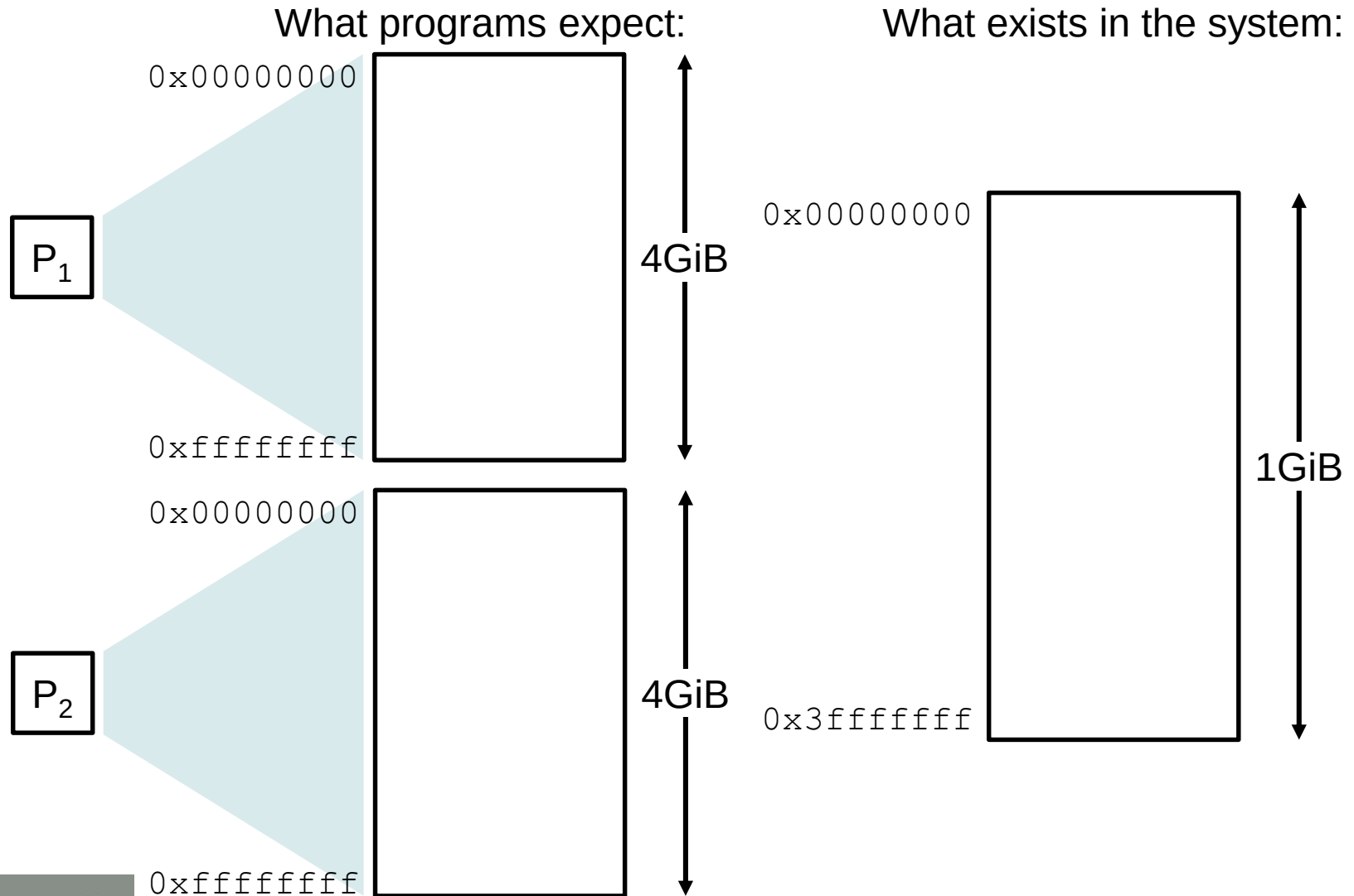
Lecture 32: Virtual Memory

CMPS 221 – Computer Organization and Design

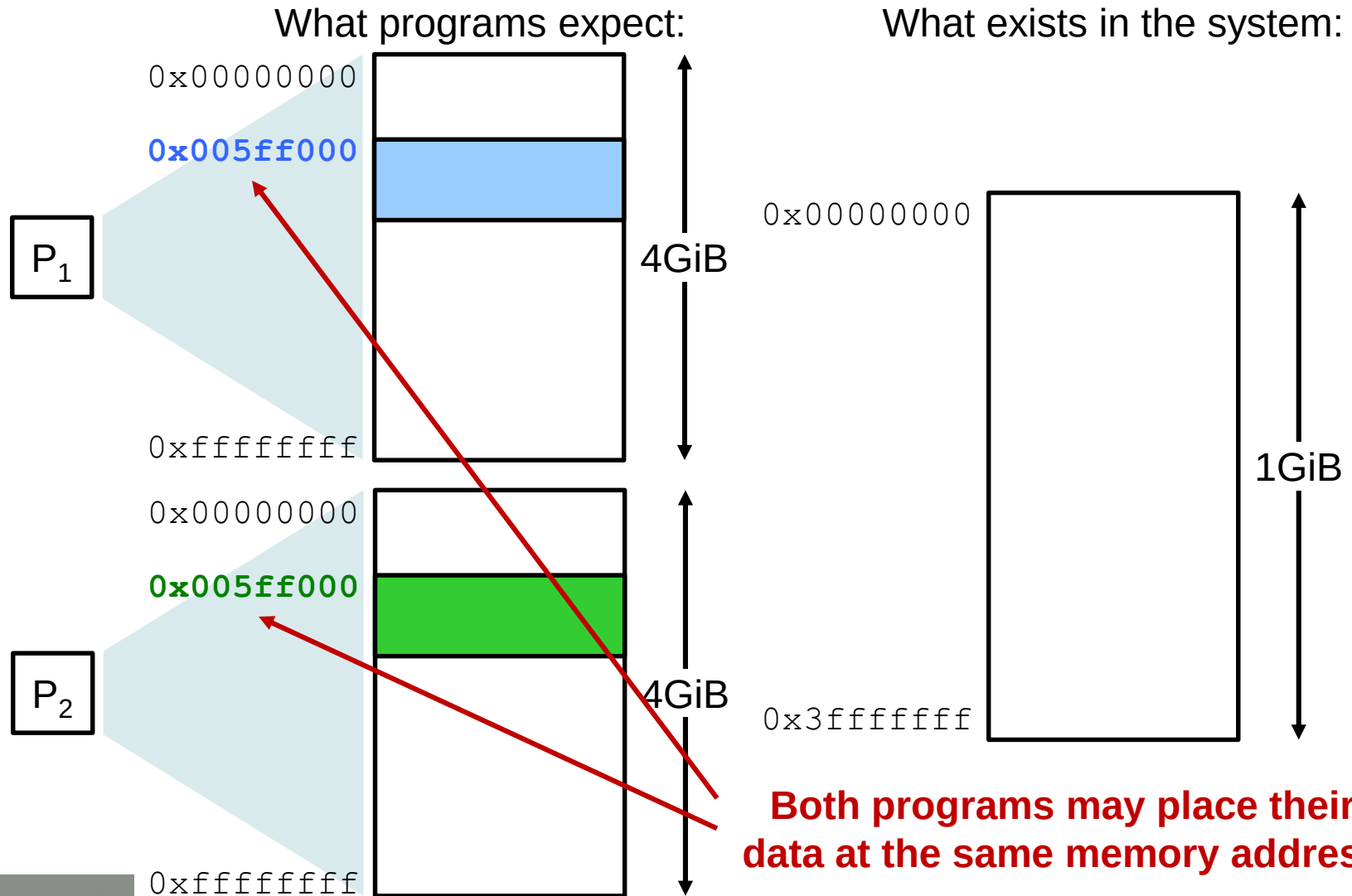
Address Spaces

- An **address space** is the range of addresses accessible by a logical or physical computational entity
- With 32-bit addresses, the size of a program's address space is 4GiB (2^{32})

Address Spaces



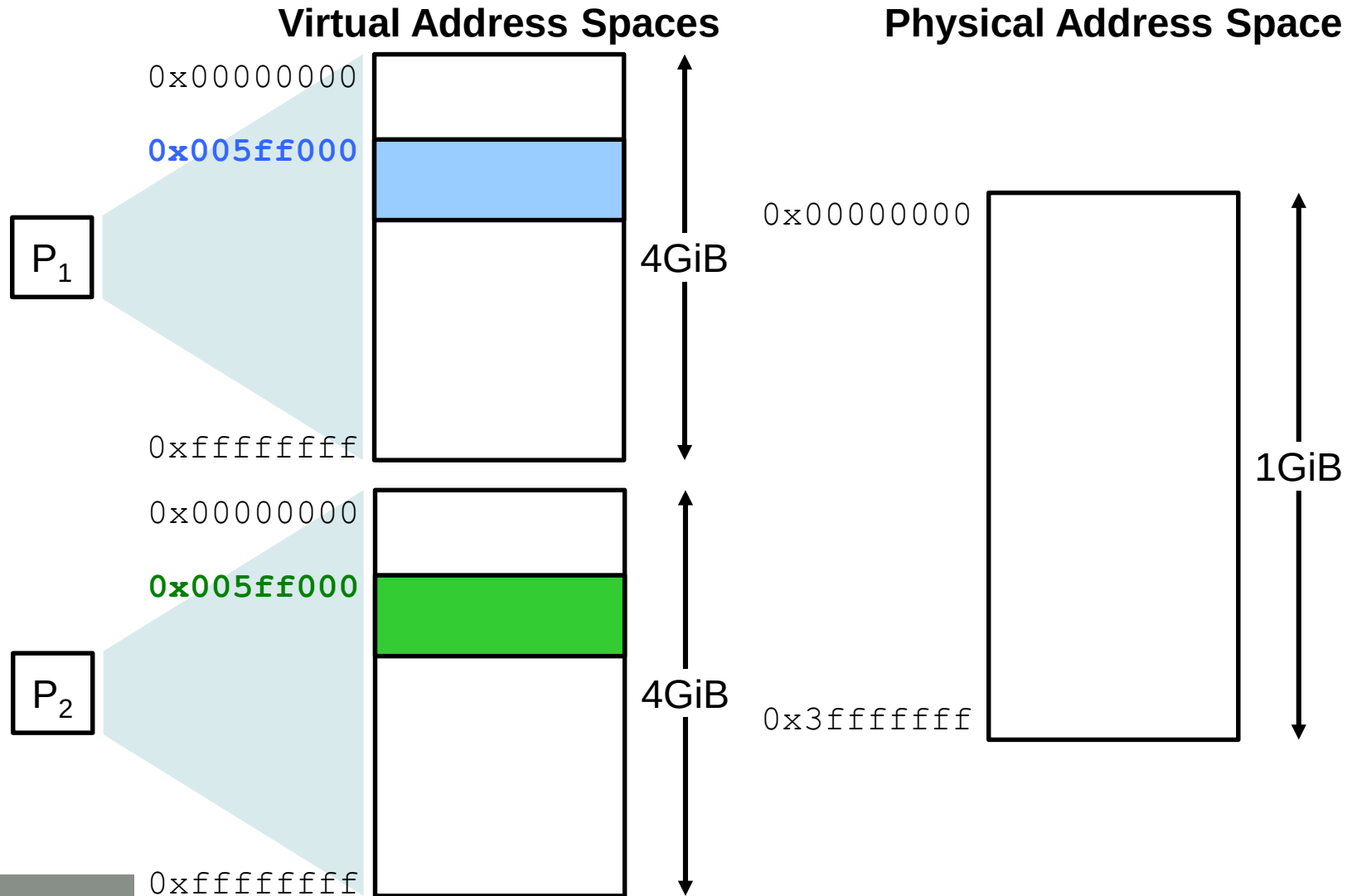
Address Spaces



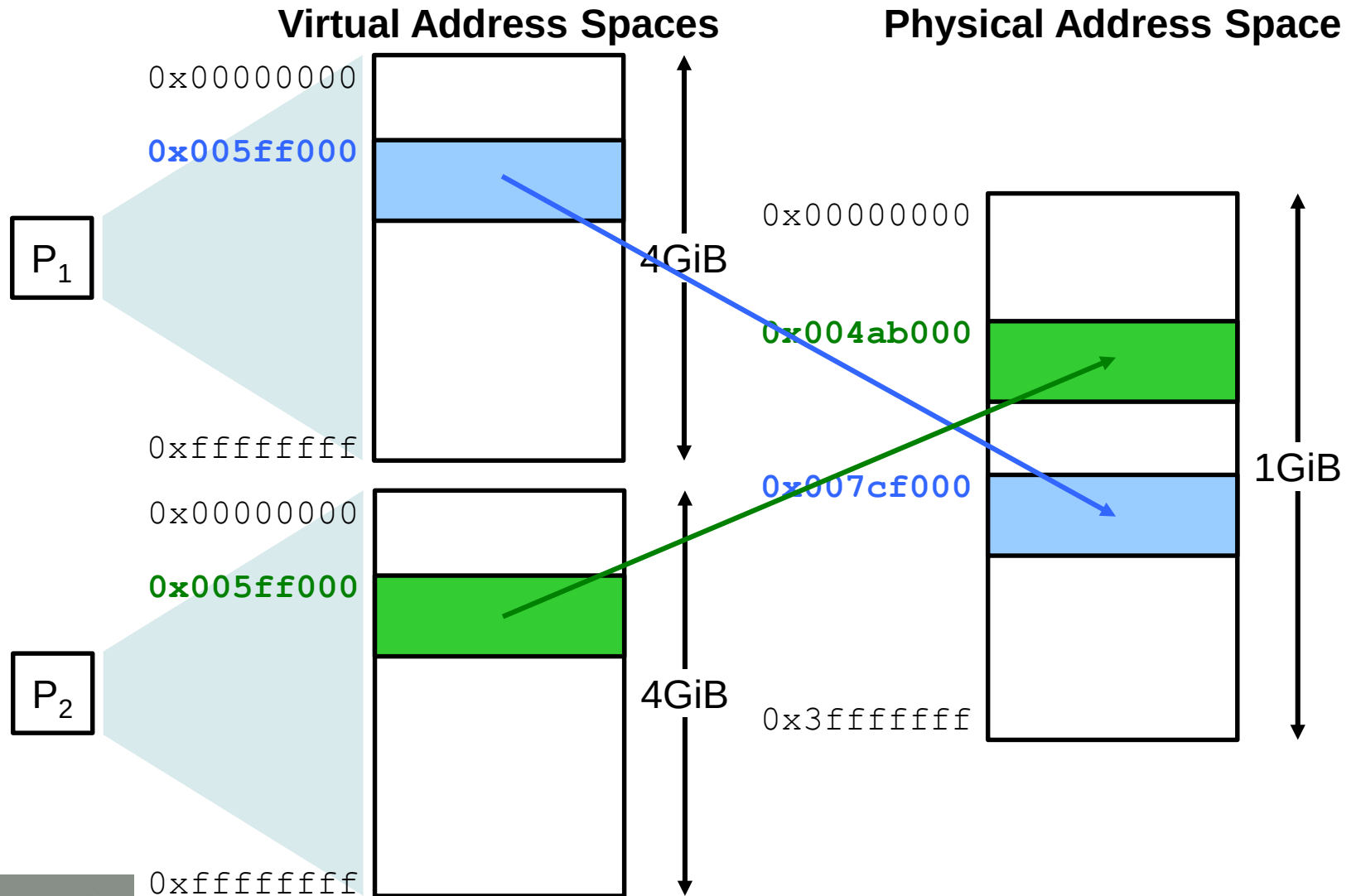
Virtual Memory

- Each program has a private logical memory holding its code and data
 - i.e., the program's **virtual address space**
- Different programs store their data in the same physical memory
 - i.e., the system's **physical address space**
- Need **address translation** to translate a virtual address to a physical address

Virtual Memory



Virtual Memory

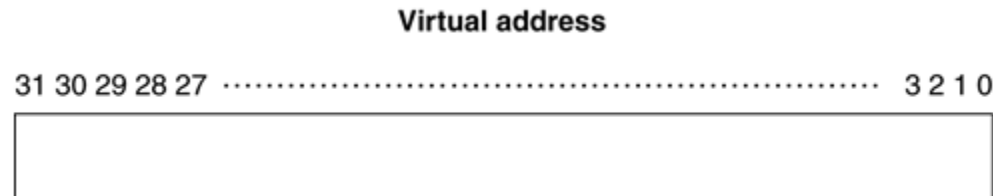


Address Translation

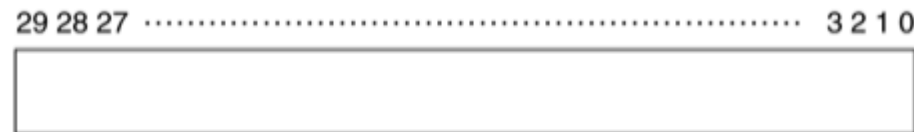
- A *block* of main memory is called a **page**
 - Serves as the unit of mapping between virtual and physical address spaces
 - Usually large (e.g., 4 KiB)
- A memory address can be divided into:
 - **Page offset**: identifies data within the page
 - The same across the virtual and physical address
 - Similar to the *block offset* in caches
 - **Page number**: identifies the page itself
 - The part that needs to be translated
 - Similar to the *tag* in caches

Address Subdivision

- Example:
 - 4 KiB (2^{12}) page size
 - 4 GiB (2^{32}) of virtual memory
 - 1 GiB (2^{30}) of physical memory



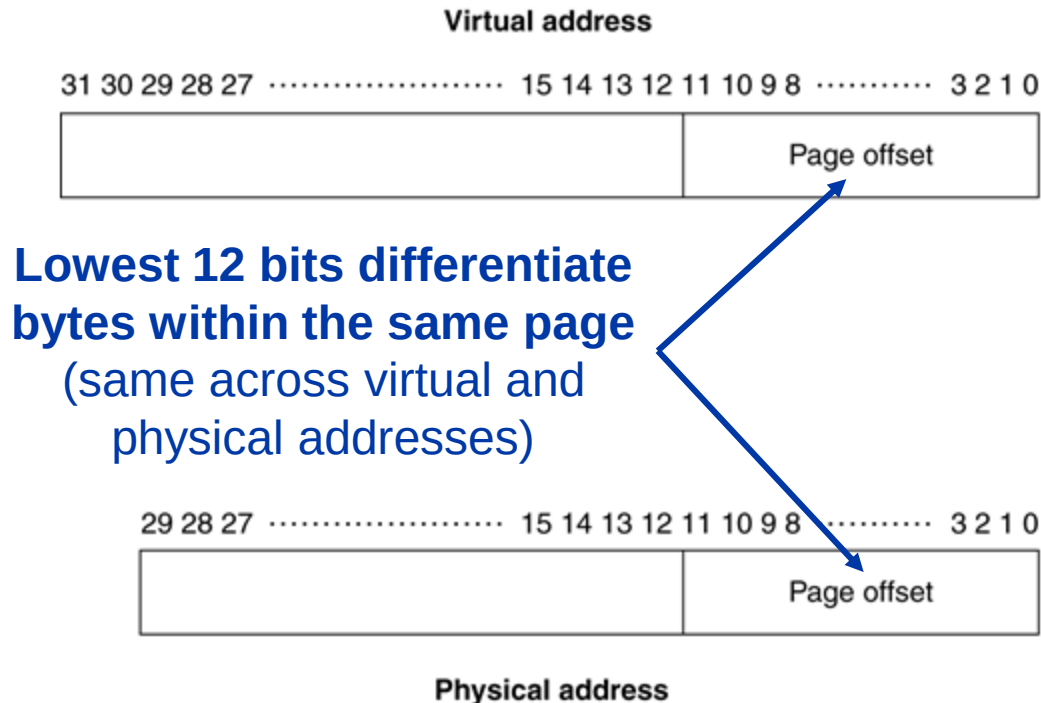
Need to translate a 32-bit address
to a 30-bit physical address



Physical address

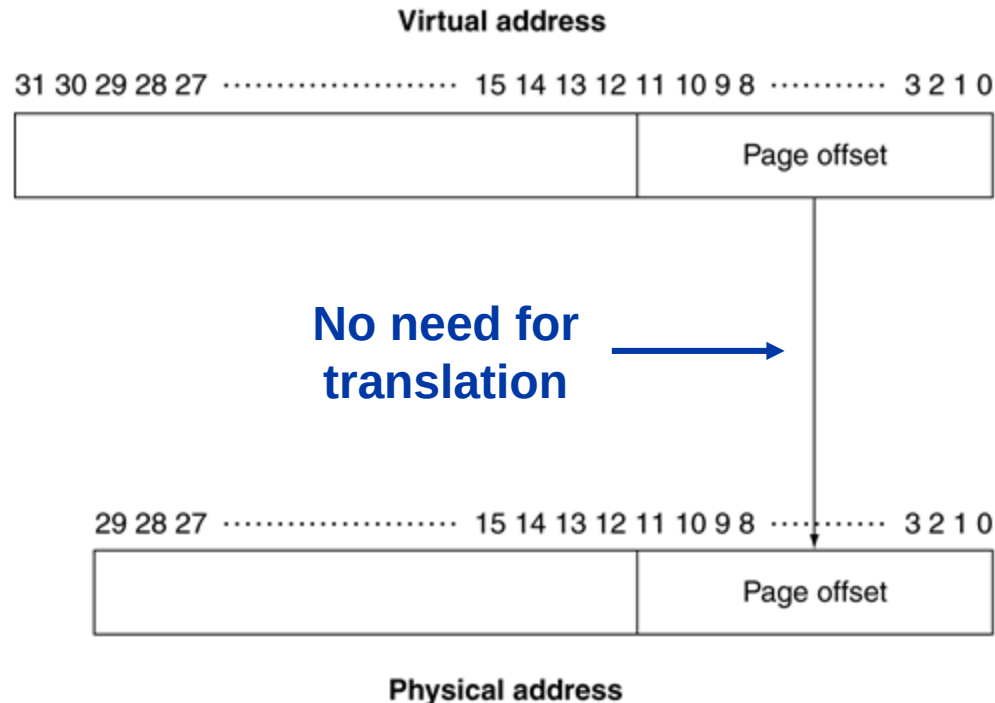
Address Subdivision

- Example:
 - 4 KiB (2^{12}) page size
 - 4 GiB (2^{32}) of virtual memory
 - 1 GiB (2^{30}) of physical memory



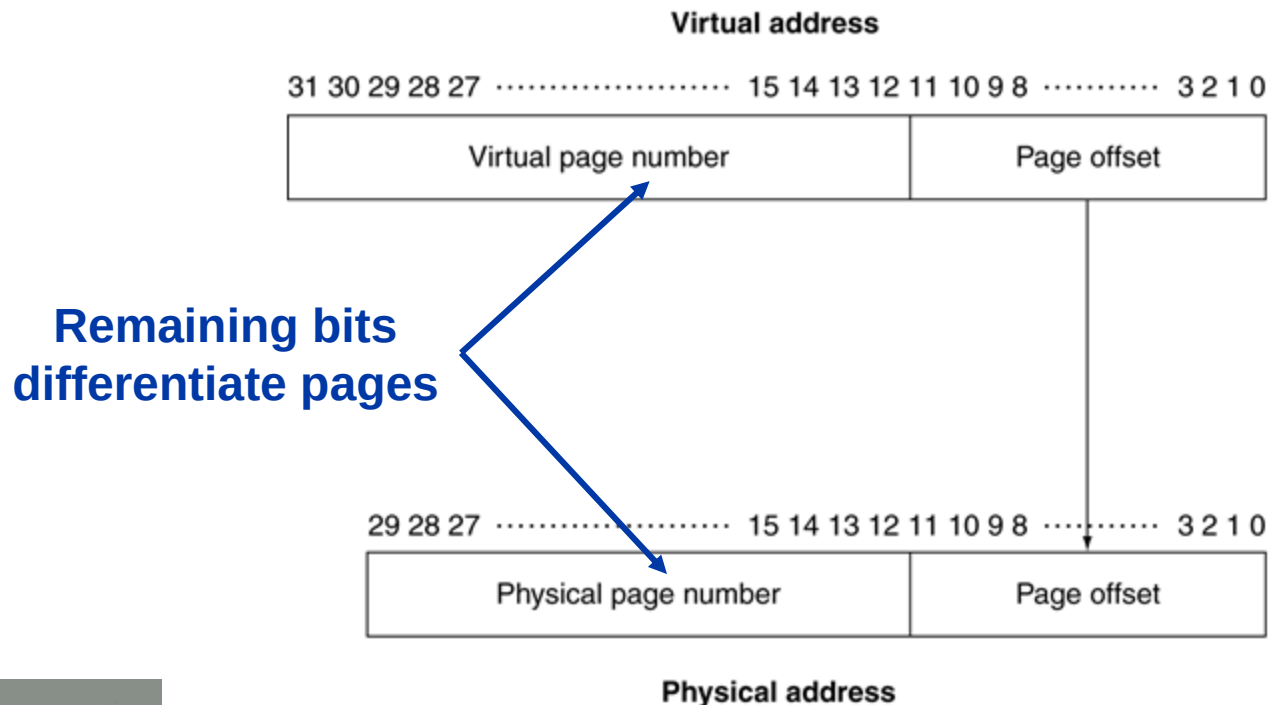
Address Subdivision

- Example:
 - 4 KiB (2^{12}) page size
 - 4 GiB (2^{32}) of virtual memory
 - 1 GiB (2^{30}) of physical memory



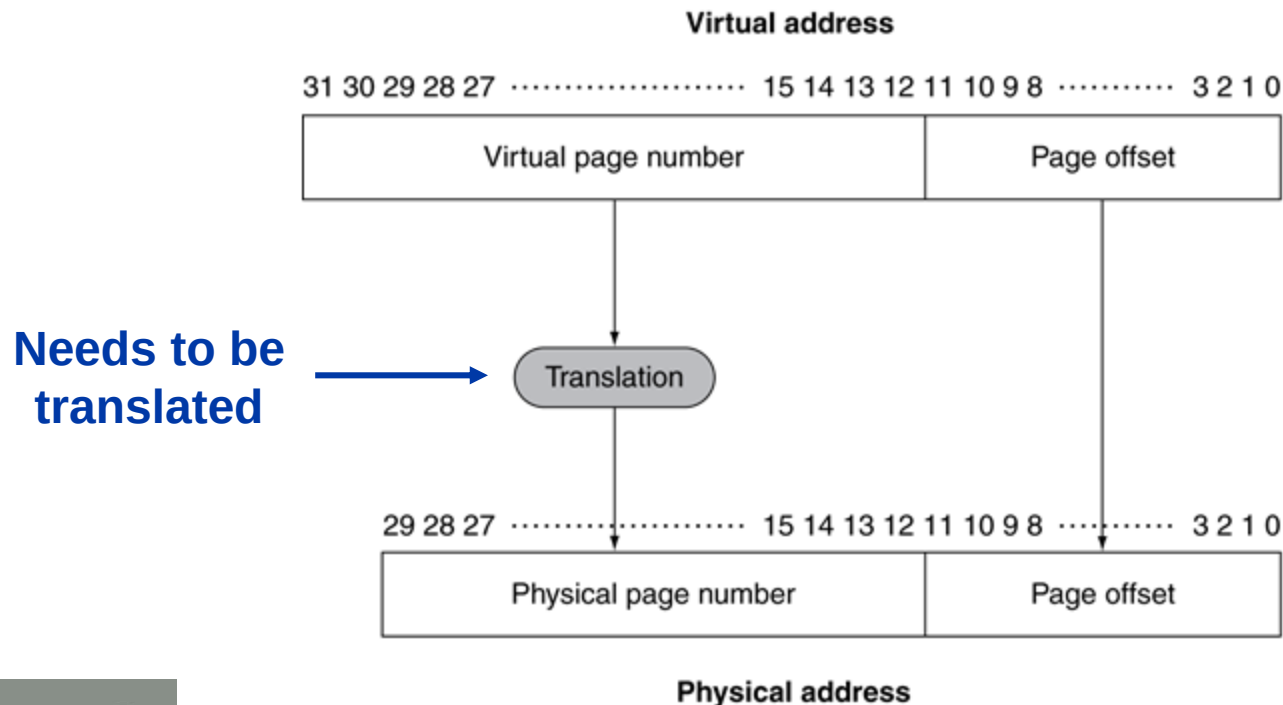
Address Subdivision

- Example:
 - 4 KiB (2^{12}) page size
 - 4 GiB (2^{32}) of virtual memory
 - 1 GiB (2^{30}) of physical memory

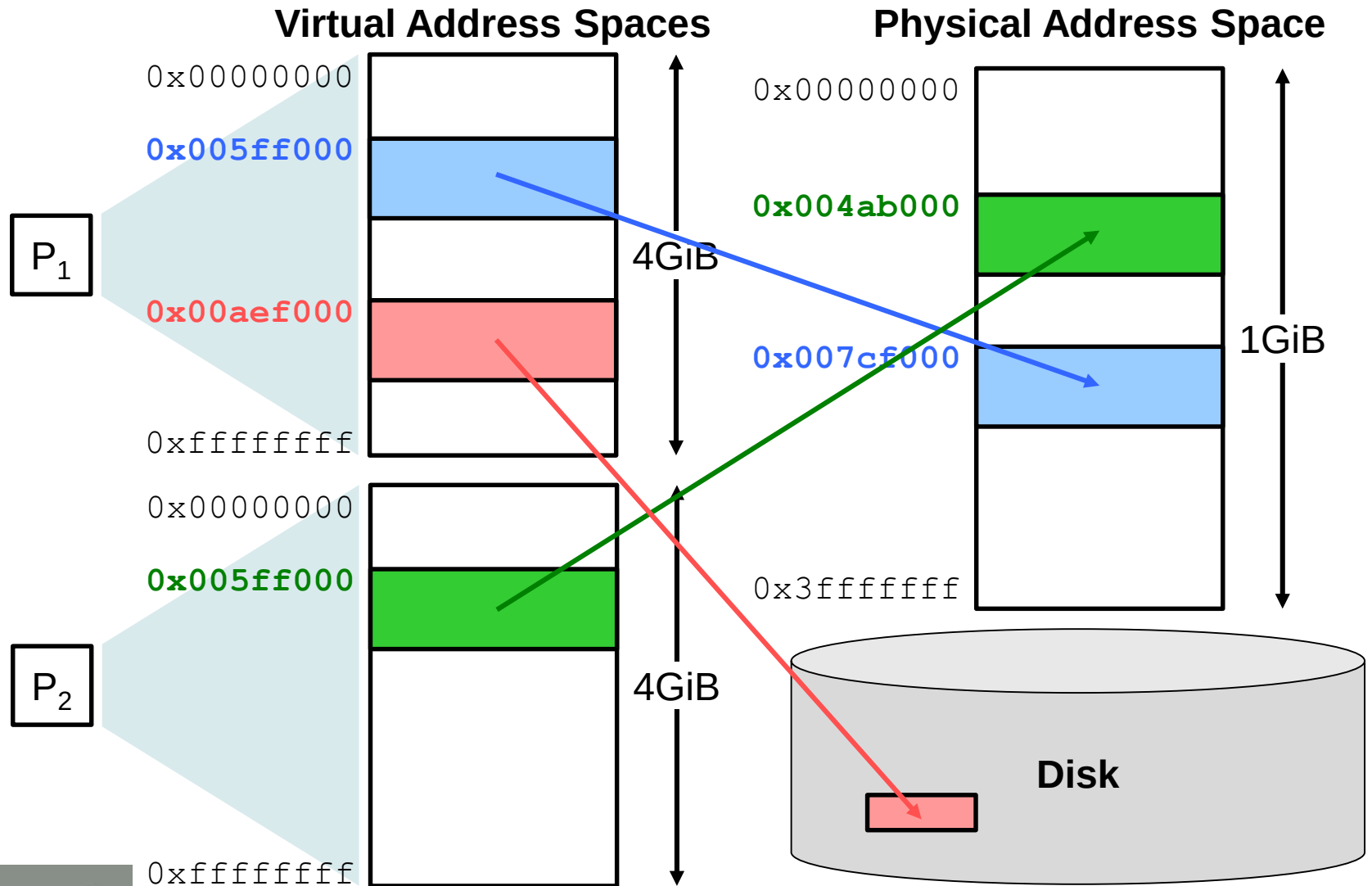


Address Subdivision

- Example:
 - 4 KiB (2^{12}) page size
 - 4 GiB (2^{32}) of virtual memory
 - 1 GiB (2^{30}) of physical memory



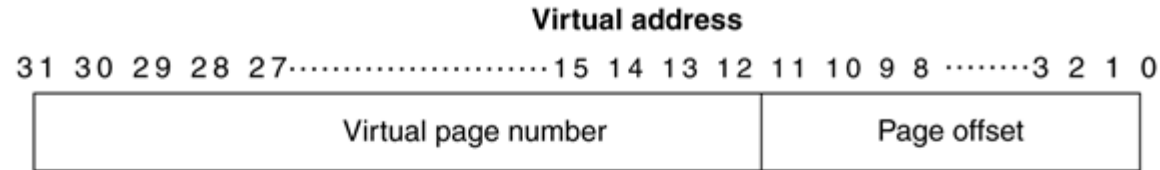
Page not in Memory



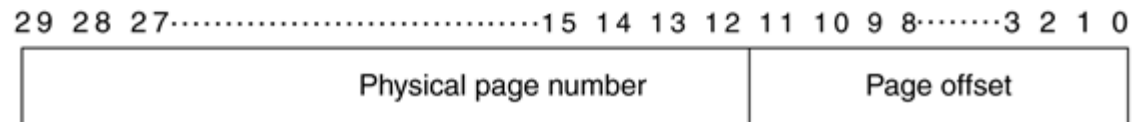
Page Fault

- A **page fault** is when a page is not found in memory and needs to be fetched from disk
 - Takes millions of clock cycles
 - Handled by the operating system
- Minimizing page fault rate
 - Large page sizes (e.g., 4 KiB)
 - Fully associative placement
 - Smart replacement algorithms
 - Can afford to run smart algorithms in software because penalty is so high

Address Translation

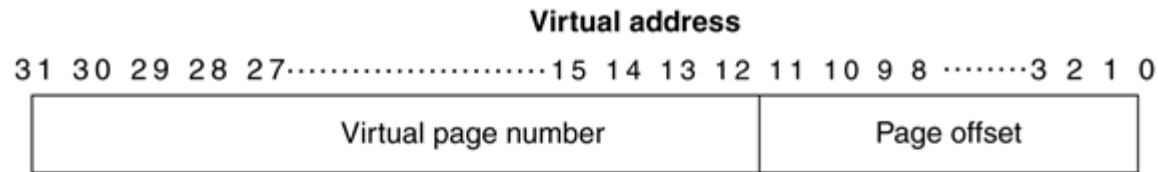


Need to translate a 32-bit address
to a 30-bit physical address



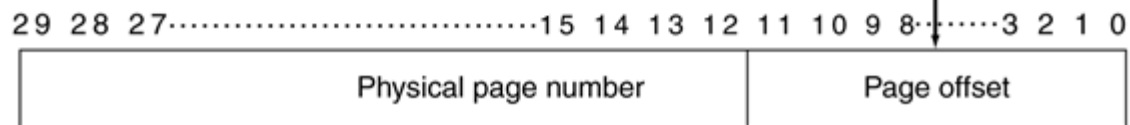
Physical address

Address Translation



How to translate a 20-bit
virtual page number to an
18-bit physical page number?

**Page offset
remains
the same**

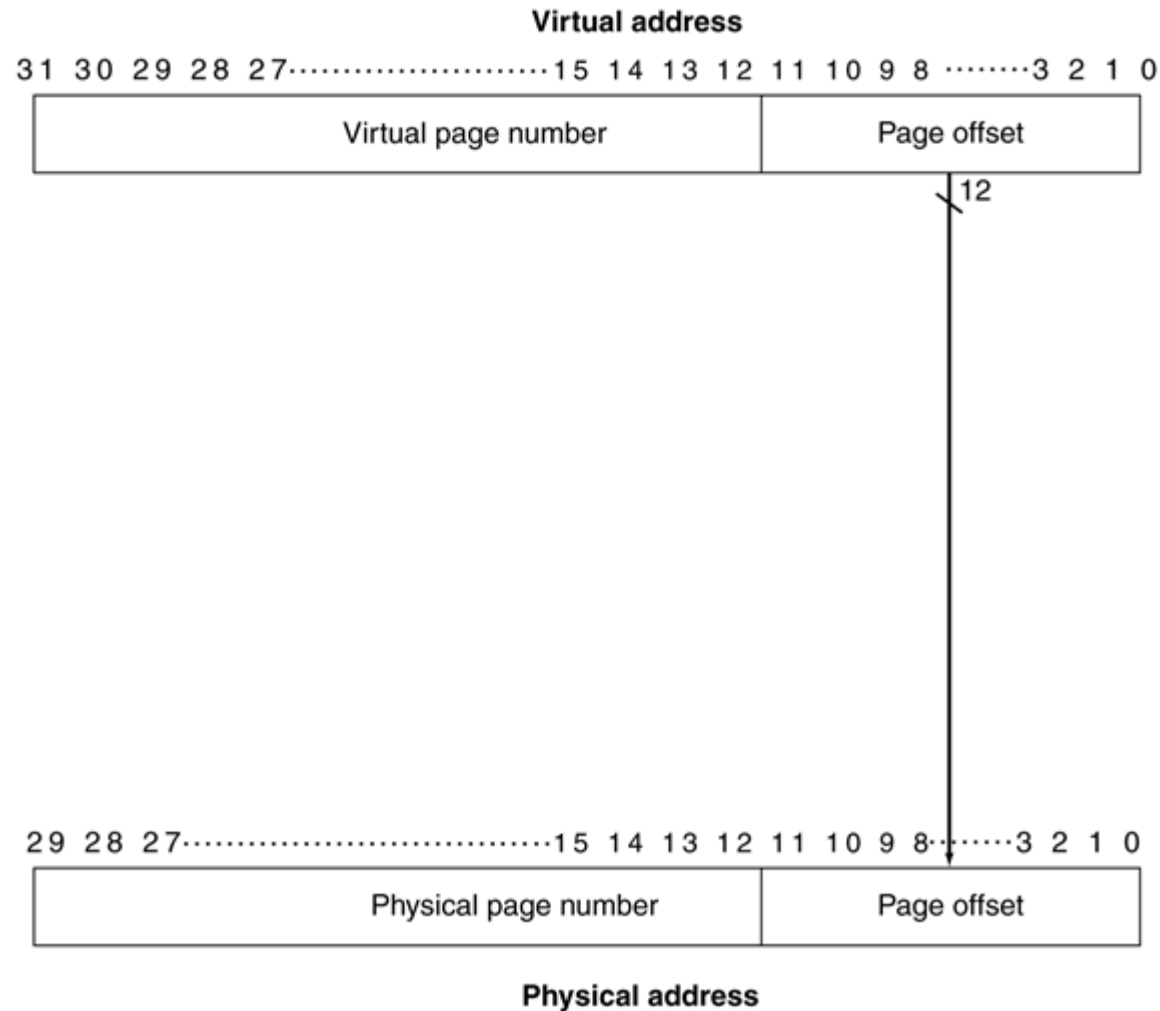


Physical address

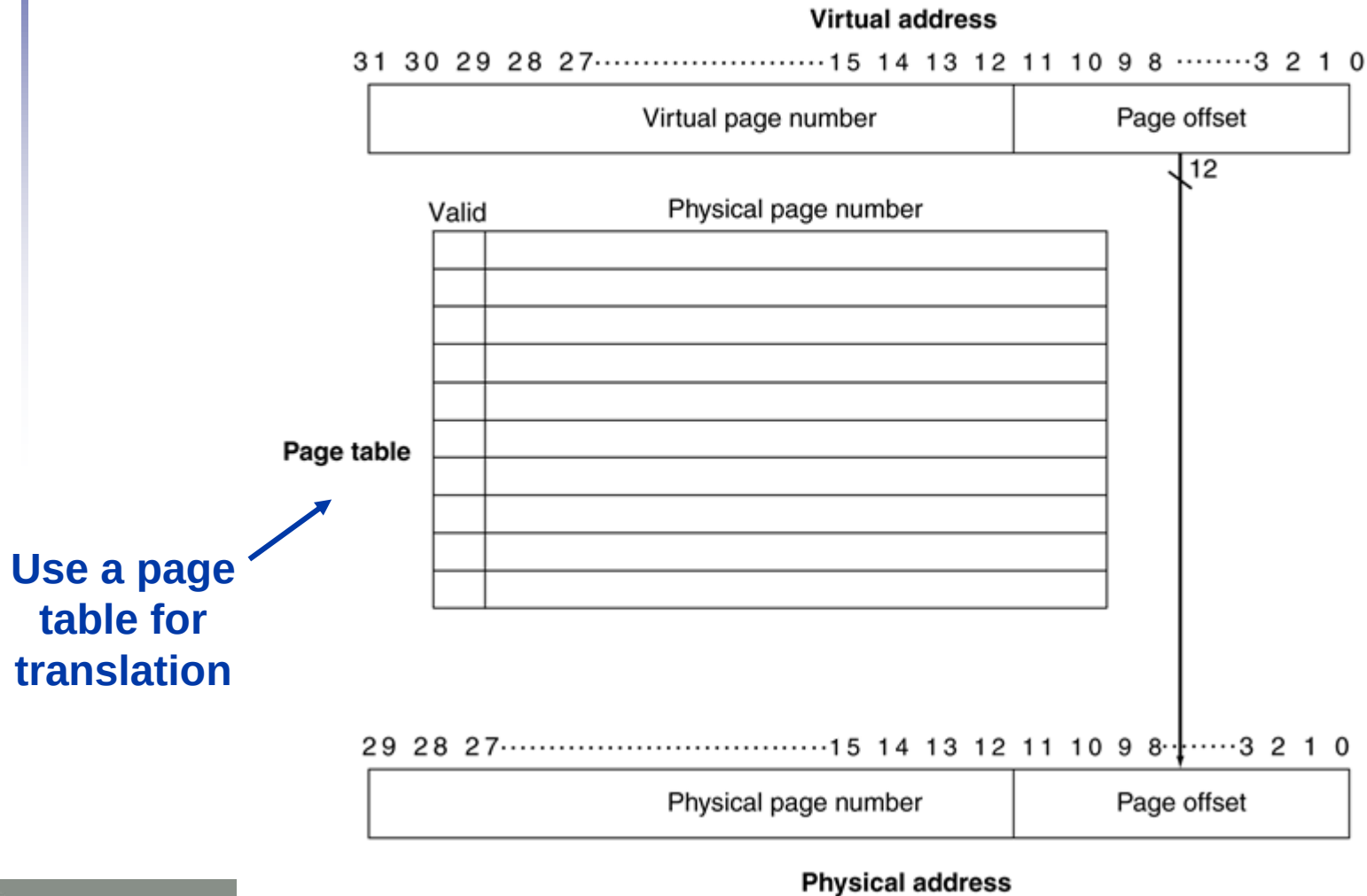
Page Tables

- Address translation uses a **page table**
 - Array of **page table entries** (PTEs)
 - Indexed by the **virtual page number** (VPN)
 - Contents of a PTE:
 - **Valid bit** (specifies if page is present in memory)
 - If page is present in memory
 - **Physical page number**, status bits (discussed later)
 - If page is not present
 - Reference to the page's location on disk (in **swap space**)

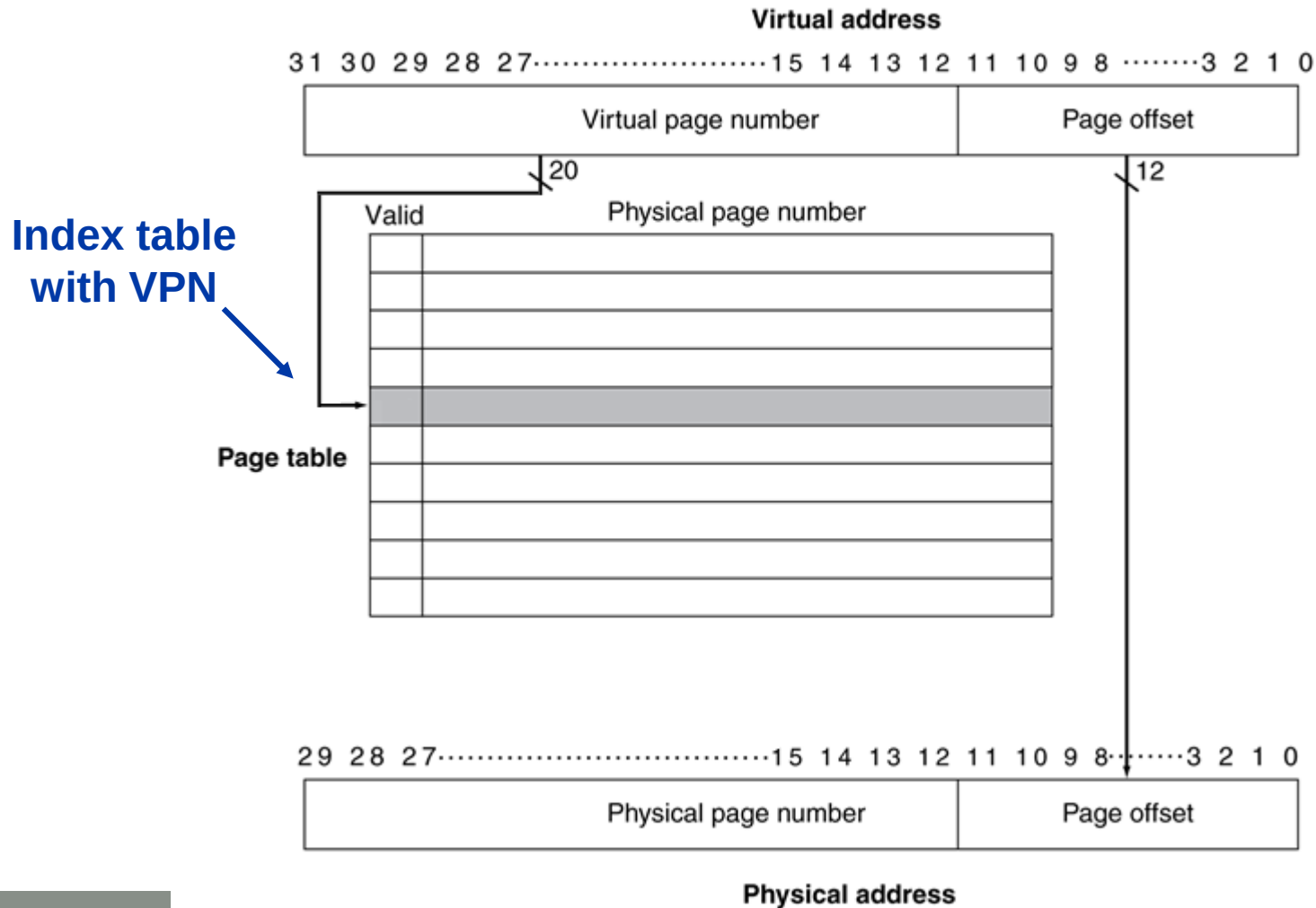
Translation Using a Page Table



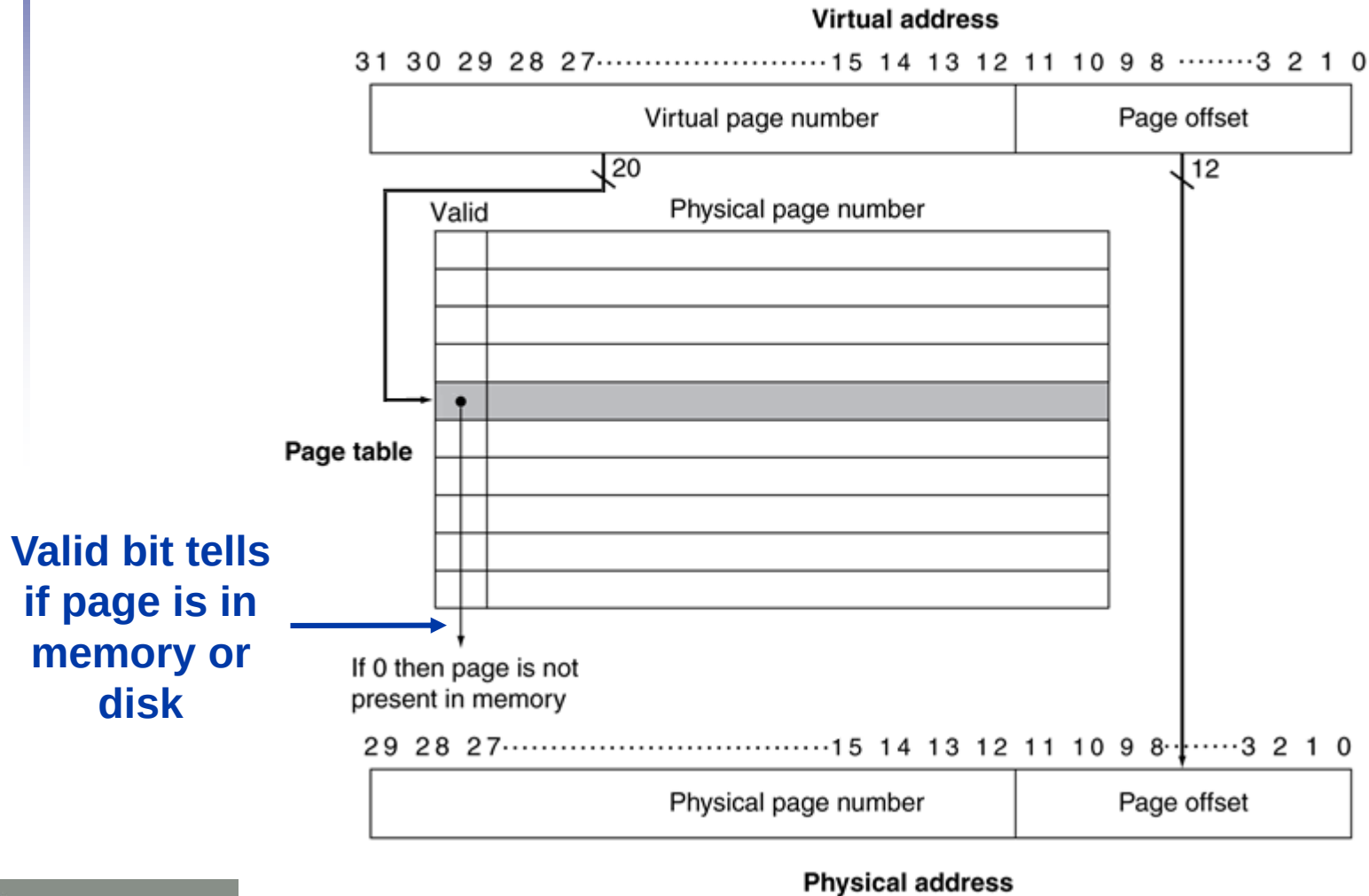
Translation Using a Page Table



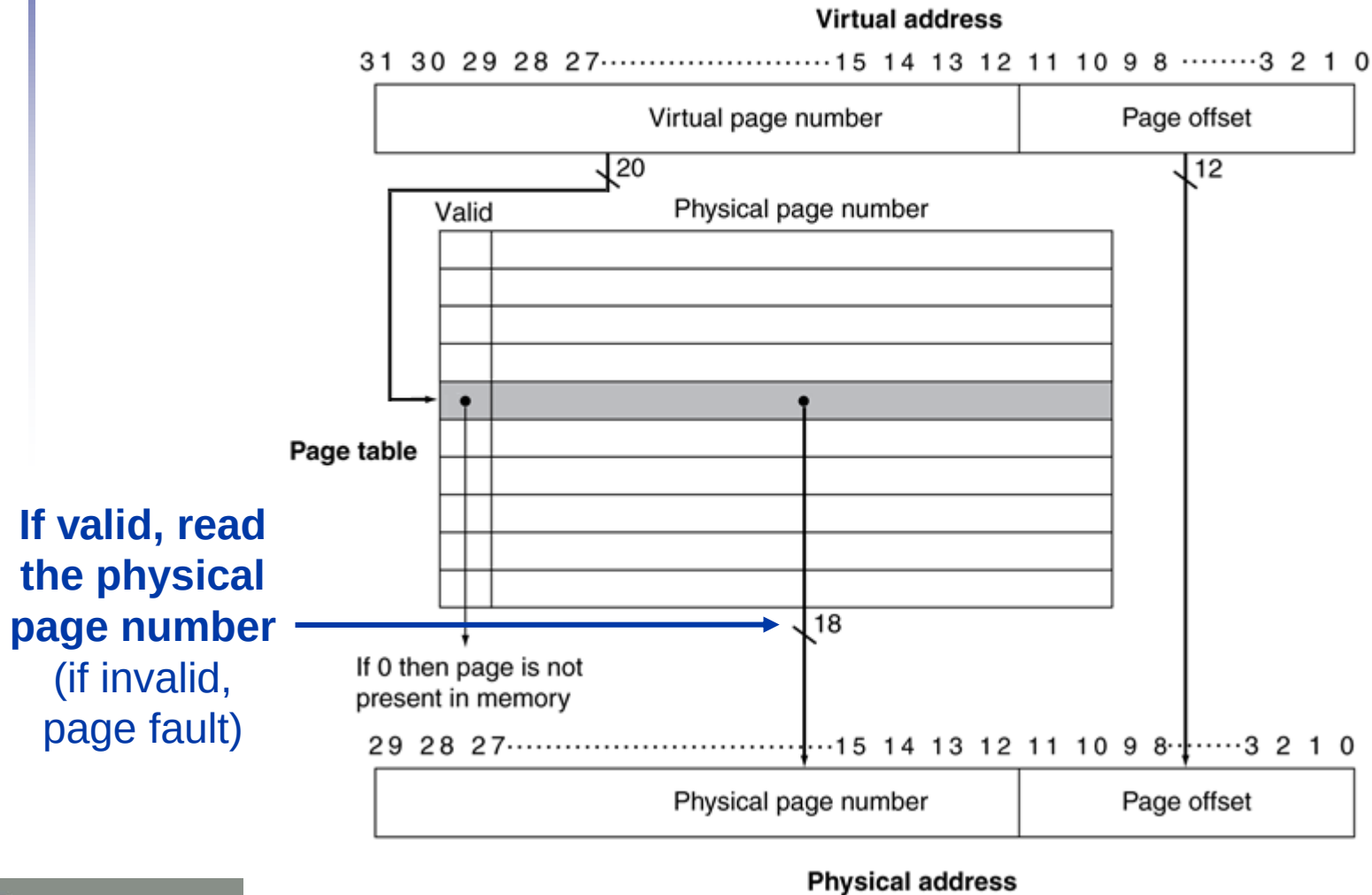
Translation Using a Page Table



Translation Using a Page Table



Translation Using a Page Table



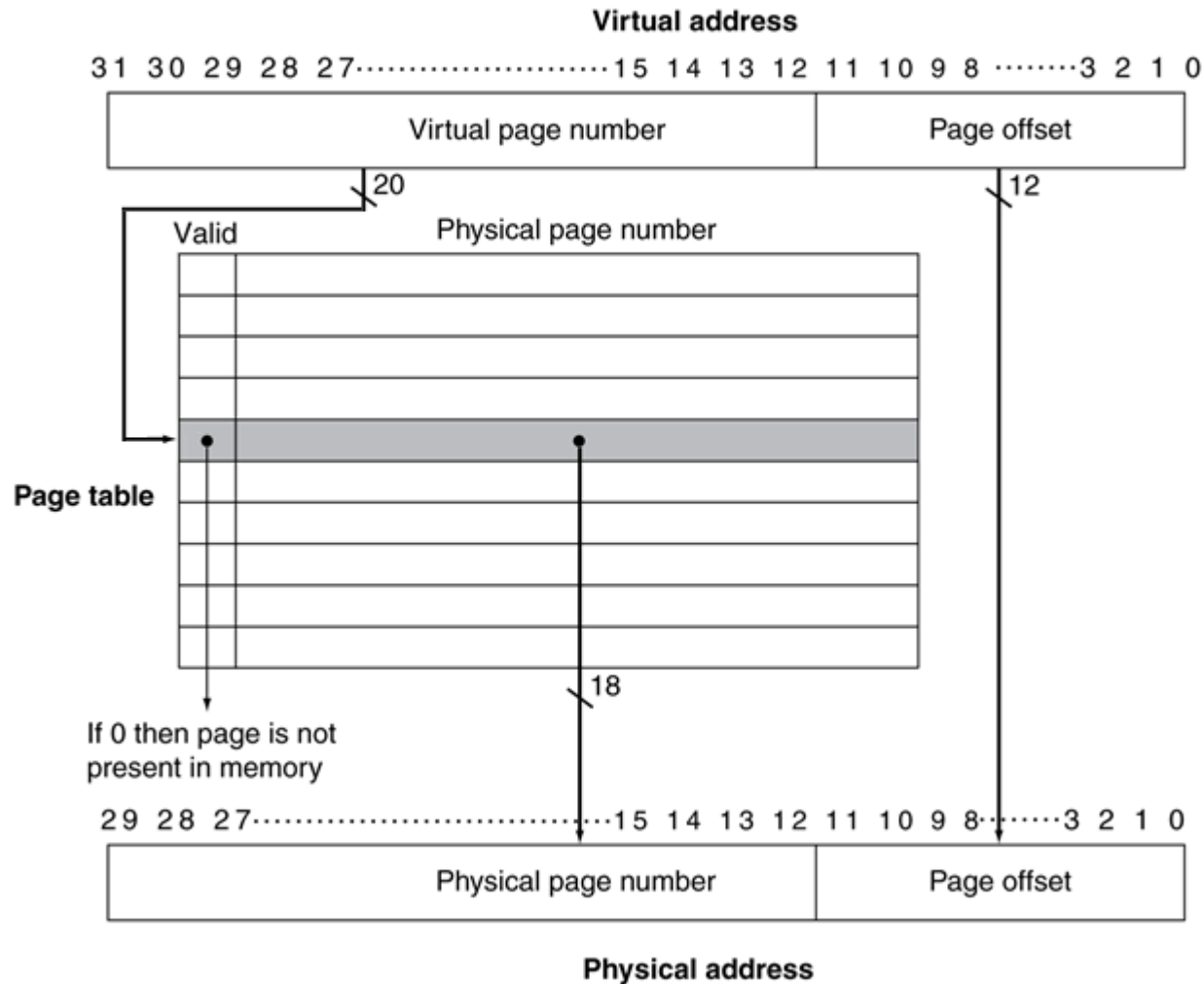
Handling Page Faults

- Actions performed on a **page fault**
 - Locate the page on disk
 - If physical memory is full, evict a page
 - If evicted page is dirty, write it back to disk first
 - Read the new page into memory
 - Update the page table
 - Resume execution from the instruction that caused the page fault

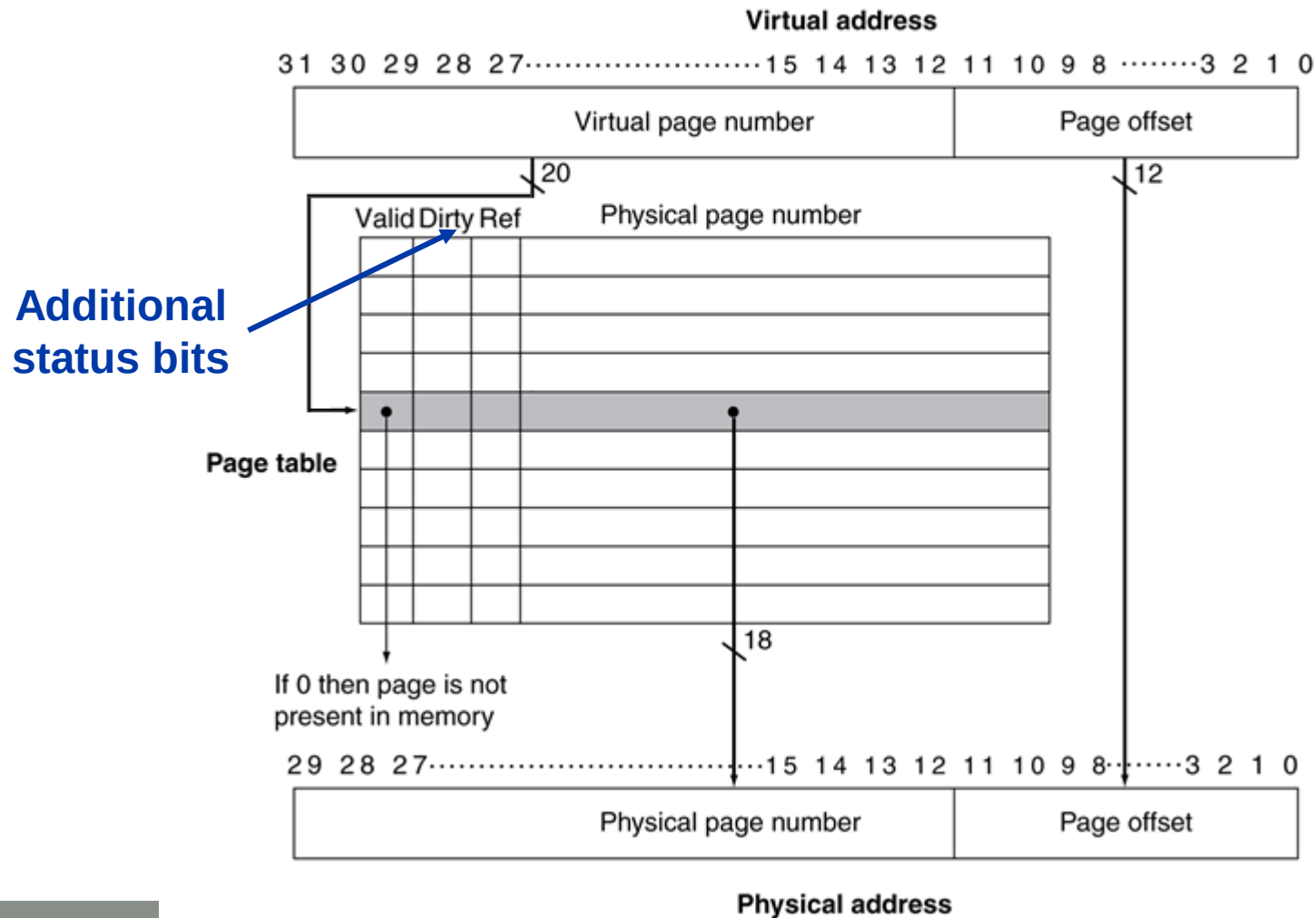
Status Bits

- Marking dirty pages
 - Disk writes take millions of cycles
 - Write through is impractical so use write-back
 - Set **dirty bit** in PTE when page is written
- Replacement policy
 - Approximate least-recently used (LRU) policy
 - Set **reference bit** (aka **use bit**) in PTE to 1 on access to page
 - OS periodically clears reference bits to 0
 - Page with reference bit = 0 has not been recently used

Page Table with Status Bits



Page Table with Status Bits



Textbook Sections

- The content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - 5.7